

## **Abstract**

This paper examines the gap between theoretically optimized approaches and practical network engineering, showing that well-known graph algorithms did not deliver usable designs or routing behavior under real traffic, capacity, and failure conditions. This issue matters because reliability, efficiency, and cost control depend on solutions grounded in actual network behavior. The main question is why the standard optimization and shortest path methods underperform in practice and what would make them really useful to practitioners.

The analysis uses conceptual reasoning involving two examples, the design of the "cheapest" network and "optimal" routing in an existing core. It is based on simple models, figures, and protocol behavior using minimal spanning tree (MST), Dijkstra's algorithm, and OSPF/IS-IS, along with a "double-drop" heuristic and ideas like ant colony optimization (ACO).

MST designs are impractical; they have no redundancy and create too-long paths. More fundamentally, link cost is dependent on capacity needed under the selected topology and aggregate flows, which means that the cost matrix is endogenous to the solution, rendering the optimization problem ill-posed. Practitioners therefore resort to topology-aware heuristics that find local minima with unknown gaps to the true optimum. Per-pair shortest-path routing ignores routing independence; routes compete for shared links. Effective costs change individually; optimal paths generate a suboptimal plan at a network-wide level. Distributed protocols share state but cannot achieve coordination in future routing to help compute global loads, so they cannot implement true optimization. Additivity of costs (e.g., capacity similar to delay) is also questioned.

The paper argues that progress requires problem-aware modelling, implementable algorithms, and better communication between theorists and engineers. While some theory (such as shortest paths in baseline routing) is still useful, closing the gap needs a better match of metrics to goals (throughput, delay, reliability, and economics) and consideration of distributed, cooperative heuristics. The contribution is an organized explanation of how prevailing methods are insufficient and what would make optimization more realistic and usable in practice.